Memory Maze – Human Vs Ai Pathfinding Game Software Requirements Specification

Version 1.2

Virtual University

Group Id: F25PROJECTB9784

Supervisor Name: Safid Ullah Shah

Revision History

Date (dd/mm/yyyy)	Version	Description	Author
21/11/2025	1.0	Introduction of the project	BC220407371
19/05/2026	1.1	Finalizing	BC220407371
19/05/2026	1.2	Use case scenarios in Tables	BC220407371

Table of Contents

Introduction

This document provides the foundation for the *Memory Maze - Human vs AI Pathfinding Game* project.

Purpose

This document specifies the requirements for the *Memory Maze - Human vs AI Pathfinding Game*, where a human player competes against an AI rival to solve dynamically generated mazes. It is intended for developers and supervisors involved in designing, implementing, and testing the system.

Scope

Memory Maze - Human vs AI Pathfinding Game aims to develop a competitive maze-solving game where a human player competes against an AI rival. At the beginning, the maze is displayed fully for a few seconds, after which parts of it disappear. The player must rely on memory and strategy to find the exit. Simultaneously, the AI rival uses the `A*` Search Algorithm (`Manhattan Distance heuristic`) to navigate the maze efficiently.

The game tests human cognitive skills against algorithmic precision, providing engaging experience and demonstrating pathfinding, algorithm design, and AI vs human performance comparison while keeping track of results. Additional complexity is introduced by multiple maze sizes, increasing difficulty levels, and power-ups, which can hinder or help in progress.

Definitions, Acronyms and Abbreviations

- `UC`: User Characteristic (e.g. `UC1` means `user characteristic 1`)
- `FR`: Functional Requirement (e.g. `FR1` means `functional requirement 1`)
- `NFR`: Non Functional Requirement (e.g. `NFR1` means `non functional requirement 1`)
- `A*`: Path finding algorithm
- `Manhattan distance heuristic`: A calculation useful in grid based pathfinding algorithms
- `Python`: Programming language
- `PyGame`: A library which integrates with `Python`

References

- IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1993, IEEE Computer Society, 1993. Available: https://ieeexplore.ieee.org/

Overview

This document describes the

- Functional requirements
- Non Functional requirements
- Use Case Diagram
- Use Case Scenarios
- Development Methodology
- Work Plan

Specific Requirements

Functional Requirements

Maze Generation and Display

- FR1: The system shall generate random mazes using `recursive backtracking` or `Prim's` algorithm.
- FR2: The maze shall be fully visible for a defined time (e.g., 5–10 seconds).
- FR3: After the reveal time, walls shall disappear partially, requiring the player to rely on memory.
- FR4: Maze complexity (size and branching factor) shall increase with levels.

Player Controls

- FR5: The player shall move using keyboard arrow keys.
- FR6: The player shall be allowed to collect power-ups (e.g., reveal part of maze, slow down AI).
- FR7: The player shall be penalized for hitting dead ends (e.g., time penalty).

Ai Rival

- FR8: The AI rival shall use the `A*` Search Algorithm with `Manhattan Distance heuristic` to compute its shortest path.
- FR9: The AI shall update its path dynamically if obstacles appear or change.
- FR10: The AI's speed shall increase with higher levels to maintain difficulty.

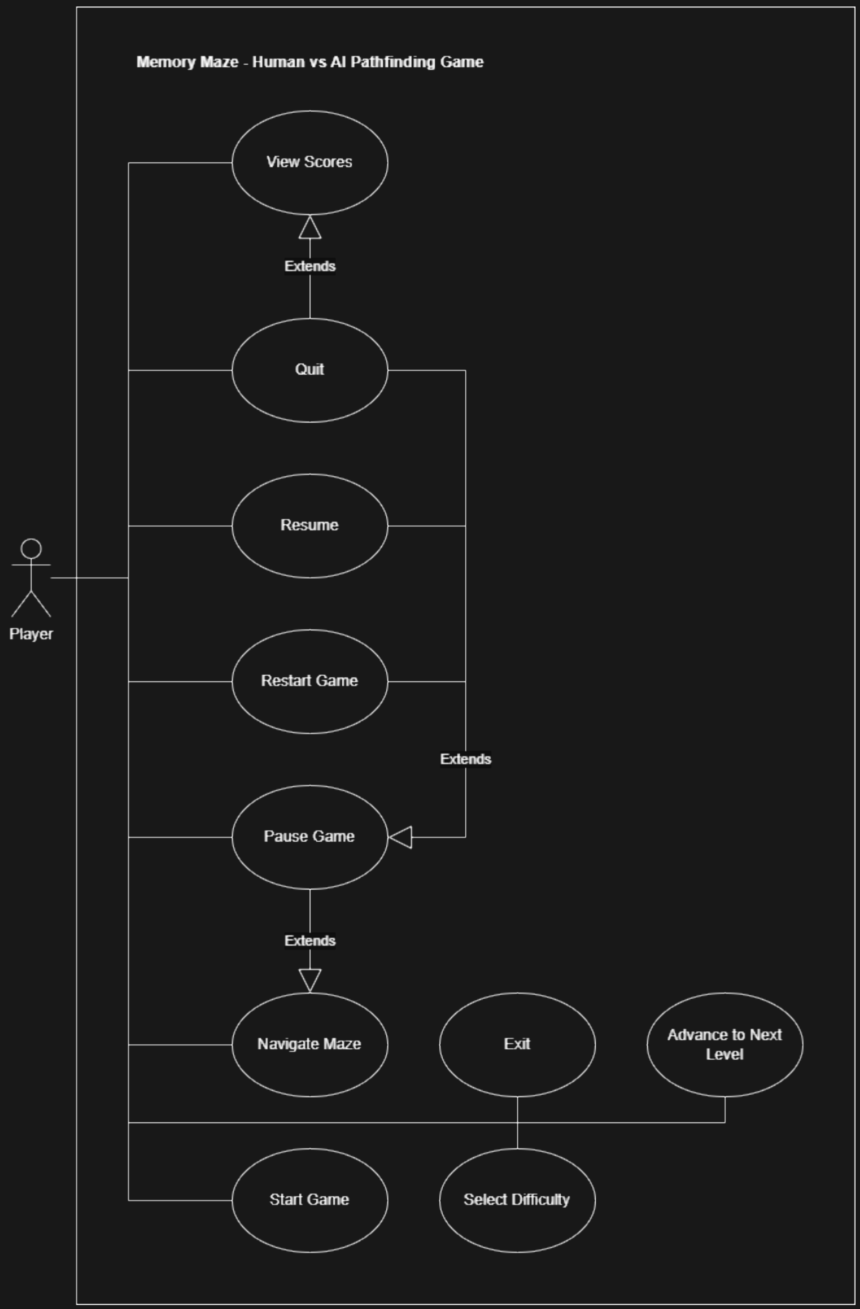
Game Logic

- FR11: The game shall run both player and AI simultaneously, updating positions in real-time.
- FR12: The game shall declare a winner when either player or AI reaches the exit first.
- FR13: The game shall maintain a scoreboard (wins, losses, fastest completion times).
- FR14: The game shall support multiple difficulty levels (small, medium, large mazes).
- FR15: The system shall allow the player to restart, pause, and exit at any time.

Non Functional Requirements

- NFA1: The game shall support 60 frames per second.
- NFA2: The game shall be designed primarily for Windows environments and is not required to support other operating systems.
- NFA3: The game shall maintain stable gameplay without crashes during a continuous session of 30 minutes.
- NFA4: The game interface shall provide clear visual feedback for player actions and game state changes.
- NFA5: The system shall support single-player gameplay only, involving one human player competing against one AI agent.

Use case Diagram



Usage Scenarios

Fields	Values
Use Case Name	Select Difficulty
Use Case ID	`UC01`
Actor	Player
Summary	This use case describes the scenario in which player selects the game difficulty.
Pre-Condition	Player must be on the main menu.
Post-Condition	Player is back on the main menu.
Extend	N/A
Uses	N/A
Normal Course of Events	Player will trigger the `Select Difficulty` button, then trigger one of the difficulty buttons (`easy`, `medium`, `hard`) and will be taken back to main menu.
Alternative Path	N/A
Exception	N/A

Fields	Values
Use Case Name	Start Game
Use Case ID	`UC02`
Actor	Player
Summary	This use case describes the scenario in which player starts playing the game.
Pre-Condition	Player must be on main menu.
Post-Condition	Player can see the maze on playing screen.
Extend	N/A
Uses	N/A
Normal Course of Events	Player triggers the `Play` button.
Alternative Path	Player can trigger `Select Difficulty` button to select difficulty, come back on the main menu and then trigger `Play` button.
Exception	N/A

Fields	Values
Use Case Name	Navigate Maze
Use Case ID	`UC03`
Actor	Player
Summary	This use case describes the scenario in which player uses arrow keys to navigate through the maze.
Pre-Condition	Maze must be visible.
Post-Condition	Player can move the avatar around.
Extend	N/A
Uses	N/A
Normal Course of Events	Player uses arrow keys to control movements.
Alternative Path	N/A
Exception	N/A

Fields	Values
Use Case Name	Pause Game
Use Case ID	`UC04`
Actor	Player
Summary	This use case describes the scenario in which player pauses the game.
Pre-Condition	Maze must be visible.
Post-Condition	Player is taken to pause menu.
Extend	Navigate Maze
Uses	N/A
Normal Course of Events	Player presses the `Escape` key and is taken to the pause menu.
Alternative Path	N/A
Exception	N/A

Fields	Values
Use Case Name	Resume
Use Case ID	`UC05`
Actor	Player
Summary	This use case describes the scenario in which player can resume the paused game.
Pre-Condition	Player must be on pause menu.
Post-Condition	Maze is visible again.
Extend	Pause Game
Uses	N/A
Normal Course of Events	Player triggers the `Resume` button and is taken back into the playing state.
Alternative Path	Player presses the `Escape` key and is taken back into the playing state.
Exception	N/A

Fields	Values
Use Case Name	Restart Game
Use Case ID	`UC06`
Actor	Player
Summary	This use case describes the scenario in which player resets the whole level.
Pre-Condition	Player must be on pause menu.
Post-Condition	Maze is visible again but level is reset.
Extend	Pause Game
Uses	N/A
Normal Course of Events	Player triggers the `Restart` button. Level is reset and game is resumed.
Alternative Path	N/A
Exception	N/A

Fields	Values
Use Case Name	View Scores
Use Case ID	`UC07`
Actor	Player
Summary	This use case describes the scenario in which player can view their scores.
Pre-Condition	Player must be on main menu or result menu.
Post-Condition	Scoreboard is visible.
Extend	N/A
Uses	N/A
Normal Course of Events	Player triggers the `Scoreboard` button on the result screen and is taken to the scoreboard menu.
Alternative Path	Player triggers the `Scoreboard` button on the main menu and is taken to the scoreboard menu.
Exception	N/A

Fields	Values
Use Case Name	Advance to Next Level
Use Case ID	`UC08`
Actor	Player
Summary	This use case describes the scenario in which player advances the levels.
Pre-Condition	Player must be on result menu.
Post-Condition	Next level is visible.
Extend	N/A
Uses	N/A
Normal Course of Events	On the result menu, player triggers the `Next Level` button and is taken to the next level.
Alternative Path	N/A
Exception	N/A

Fields	Values
Use Case Name	Quit
Use Case ID	`UC09`
Actor	Player
Summary	This use case describes the scenario in which player is taken to the main menu.
Pre-Condition	Player must be on any of the menus.
Post-Condition	Main menu is visible.
Extend	Pause Game, View Scores
Uses	N/A
Normal Course of Events	From any menu, player triggers the `Quit` button and is taken to the main menu.
Alternative Path	N/A
Exception	N/A

Fields	Values
Use Case Name	Exit
Use Case ID	`UC10`
Actor	Player
Summary	This use case describes the scenario in which player terminates the game.
Pre-Condition	Player must be on the main menu.
Post-Condition	Game terminates.
Extend	N/A
Uses	N/A
Normal Course of Events	Player triggers the `Exit` button and game is terminated.
Alternative Path	N/A
Exception	N/A

Development Methodology

Software development process often refers to the high-level process that governs the development of a software system from its beginning to its end of life – known as a methodology, model or framework.

Various methodologies have been devised, including waterfall, spiral, agile, rapid prototyping, incremental and synchronize and stabilize.

Adopted Methodology

The `iterative methodology` was chosen for the `Memory Maze - Human vs AI Pathfinding Game`.

The basic idea behind this method is to develop a system through repeated cycles (iterative) and in smaller portions at a time (incremental), allowing software developers to take advantage of what was learned during development of earlier parts or versions of the system. Learning comes from both the development and use of the system, where possible key steps in the process start with a simple implementation of a subset of the software requirements and iteratively enhance the evolving versions until the full system is implemented

Reasons for Chosen Methodology

- Features can be developed independently and integrated gradually.
- The approach allows continuous testing and debugging.
- It provides flexibility for improving gameplay experience.
- It allows developer to get familiar with the tools.

Work Plan